Predicting Performance for Co-Hosted DNNs in Frequently-Updating Environments

Sarah Shah
Computing Sciences and Mathematics
Mount Royal University
Calgary, Canada
sshah@mtroyal.ca

Yasaman Amannejad
Computing Sciences and Mathematics
Mount Royal University
Calgary, Canada
yamannejad@mtroyal.ca

Diwakar Krishnamurthy
Electrical and Software Engineering
University of Calgary
Calgary, Canada
dkrishna@ucalgary.ca

***Abstract*—Deep Neural Networks (DNNs) are often co-hosted on shared resources in time-critical applications such as fraud detection and security surveillance, where Service Level Agreements (SLAs) enforce strict response time requirements. However, response times vary with co-hosted workloads, request rates, and resource constraints. This challenge is amplified as new DNNs are frequently introduced, making it essential to predict their performance without prior co-hosting data. In this paper, we propose a novel approach to predict the response times of co-hosted DNNs in dynamic environments. Our method models interactions between DNNs using resource utilization patterns, isolated DNN behavior, and system and workload factors. Through extensive experiments, we show that our approach can accurately predict the performance of previously unseen DNNs with minimal prior information, achieving an average error of 13.6% across diverse configurations. These findings provide insights that can facilitate dynamic resource allocation and scheduling in multi-tenant DNN systems, which can help reduce SLA violations.**

***Index Terms*—Deep Neural Networks (DNNs), Co-hosted DNNs, Performance Prediction, Response Time, Multi-tenancy**

I. Introduction

As Deep Neural Networks (DNNs) are increasingly deployed across applications, efficient resource allocation becomes critical. DNNs are typically deployed on ML serving platforms, which host trained models and expose them via APIs for use. Due to resource and cost constraints, multiple DNNs are often co-hosted on shared hardware. In this paper, *co-hosting* refers to deploying multiple DNNs on the same physical machine so that they share system resources such as processors, memory, and cache. This makes it essential to understand their *performance interactions*, that is, how contention for shared resources affects the response time of each DNN. This is particularly important for maintaining Service Level Agreements (SLAs), which enforce strict response time limits in time-sensitive settings. For instance, in fraud detection systems, DNNs must classify transactions within tight deadlines to prevent costly consequences.

Ensuring SLA compliance while avoiding resource overuse is challenging. DNNs are computationally intensive and sensitive to fluctuating request rates. Co-hosting further complicates this by introducing *interference* from shared resource usage, making response time behavior harder to predict. In high-stakes environments, accurately predicting DNN performance based on system configuration, request rates, and DNN characteristics is key to reliable deployment.

While prior work has improved DNN efficiency through techniques like quantization [1], pruning [2], and restructuring [3], these focus on optimizing individual models. Characterization studies [4]–[8] provide valuable benchmarks but lack generalizability, and existing prediction approaches [9]–[11] remain limited in scope or accuracy. Research on co-hosted DNN performance is still limited [12]–[14]. One study [8] proposed a unified model for predicting performance across unseen configurations, later extended in [15] to capture cross-DNN interference using isolated behavior (i.e., when a DNN runs alone). While effective, these methods require substantial prior data per DNN, limiting applicability to new DNNs.

This requirement for prior per-DNN performance data defines the main research gap addressed in this paper. In practice, businesses deploy new DNNs frequently, especially in Continuous Integration/Continuous Deployment (CI/CD) workflows, leaving limited opportunity to collect extensive prior performance data before deployment. In such settings, predicting the behavior of a newly introduced DNN in a co-hosted environment becomes essential to avoid cost and time losses. By a *newly introduced DNN*, we mean a DNN for which no prior co-hosting performance data is available in the target deployment platform. The central question, therefore, is whether co-hosted response time can be predicted for such DNNs using only system configuration, request rates, a small amount of isolated information, and resource usage estimates, rather than extensive prior co-hosting data.

This paper examines whether response times can be accurately predicted for unseen DNNs, co-hosted with another DNN, using request rate, system configurations, a quick peek into their isolated behavior, and predictions for resource usage, without extensive prior data on all co-hosted DNNs. More specifically, this paper addresses two key research questions:

- **RQ-1**: Is the response time of a DNN under a given configuration of resources and request rates identifiable from the resource utilization of that DNN and the DNN co-hosted with it?
- **RQ-2**: Can the response time of a previously unseen DNN be accurately predicted in a co-hosted setting without prior co-hosting performance data?

This paper makes three contributions. First, if two DNNs are co-hosted, it formulates the problem of predicting response

times for unseen DNNs in co-hosted environments with minimal prior performance information about the unseen DNN. Second, it examines whether utilization signals from the target and co-hosted DNNs can capture response behavior across resources and request rates. Third, it proposes and evaluates a prediction technique combining system configuration, request rates, limited isolated behavior, and predicted resource utilization to estimate response time under co-hosting.

II. Related Work

Several studies like Clockwork [16], Ebird. [17], Gillis [18], and MArk [19] explore ML serving platforms, improving or stabilizing response times via controlled request rates and hardware [16], specialized execution and GPU memory banks [17], model partitioning [18], and hardware accelerators [19]. However, they reveal a gap between efficient serving systems and predictive performance techniques.

Though limited, prior work has explored performance modeling for ML systems. Li et al. [11] use Deep Reinforcement Learning and Bayesian Optimization to optimize inference under cost and SLA constraints. Cai et al. [10] allocate GPUs using pre-measured model profiles, achieving 94.5% SLA compliance with up to 17.0% error for image classification workloads, while noting the need for task-specific profiling. Fu et al. [9] build a predictive model for 13 ML and non-ML jobs across varying inputs and resources, but report errors up to 128.6%. Finally, Shah et al. [8] propose a hybrid method to predict DNN performance under varying rates and resources, achieving 98.1% SLA classification accuracy and 9.1% response time error, though assuming DNNs operate in isolation, unlike typical deployments.

Many DNN-based systems host multiple DNNs performing different tasks, which share underlying resources. LeMay et al. [20] note that efficient resource allocation in DNN multitenancy (i.e., cohosting DNNs) can save costs by up to 12%. However, co-hosting can interfere with response times, complicating performance predictions. The impact of co-hosted DNNs on response times is underexplored in the literature.

Ghodrati et al. [14] address this with Planaria 1, a dynamically fissioning architecture that splits into molecular DNN inference engines for hosting multiple DNNs on a single machine, outperforming another framework called Versa-DNN [13] in throughput and SLA satisfaction. Shah et al. [15] contribute to the research on co-hosted DNN performance by proposing an ML-based technique that predicts if a pair of DNNs under a specific resource-request rate configuration will complete execution within pre-defined SLA targets with a 90.2% overall accuracy. They show that their performance model can offer predictions for new, previously unseen deployment configurations with a small drop in their model's accuracy. While this work provides valuable practical insight, the approach requires prior performance data for DNNs already in the system to train the model, limiting generalization for realistic deployments with frequently changing DNNs.

The literature reveals a lack of research on the prediction of performance for DNN deployments under realistic conditions, such as varying resources, DNN types, request rates, and resource sharing. Existing studies fall short in providing accurate and comprehensive predictive insights for dynamic environments. There is a notable lack of work on predicting performance for new DNNs, despite the increasing frequency of new DNNs being introduced into existing systems to add additional automated decision-making. Addressing this gap, we propose a performance prediction approach guided by **RQ-1** and **RQ-2**, which can help identify response times for previously unseen DNNs. This will help system operators estimate scheduling, moderate requests, and allocate resources to ensure that new DNNs do not struggle with poor performance due to lack of prior performance data.

III. Methodology

Two DNNs running together on the same system are likely to share resources. Normally, the two DNNs are performing different tasks, and are thus experiencing different request rates and consequently resource usage. We conduct our analysis by considering one DNN to be the DNN that is newly arriving to the system and the one we are predicting performance for, thus referred to as DNN_{new}, receiving a request rate of $Rate_{new}$; and the other DNN to be already running on the system, and thus referred to as DNN_B, receiving requests at $Rate_B$.

To achieve our main goal of estimating performance for DNNs with no prior performance data, we first expand on the research questions listed in Section I to understand the motivation driving this work:

1) To answer **RQ-1**, we investigate if a new DNN (DNN_{new}), deployed with a co-hosted DNN (DNN_B), exhibits similar response times regardless of the co-hosted DNN, provided that the resource utilization of the two DNNs remains the same.
2) Using **RQ-1**, we address **RQ-2**: can we predict DNN_{new}'s response time without prior co-hosted performance data?

We hypothesize that if the response time of a DNN depends on the utilization of resources caused by that DNN and the one co-hosted with it, independent of the co-hosted DNN's identity, then it can be modeled using these utilizations and other deployment variables. This could potentially remove the need for prior performance data, enabling predictions for new DNNs. We present a pipeline based on this hypothesis.

A. Experimental and Modeling Parameters

We analyze parameters affecting co-hosted DNN response times to assess prediction feasibility for an unseen DNN (DNN_{new}). Specifically, we examine three groups. We start with *System and Workload Parameters*, including *Rate*, which quantifies the average rate at which the DNN is receiving requests, as well as *Processors* and *Memory*, which quantify the system resources allocated to each configuration.

The second category is *Baseline Parameters*. As will be shown shortly in Section V-A, DNNs exhibit distinct isolated behaviors. We hypothesize that *quick* baseline insights, capturing sensitivity to request rates and resources, can indicate

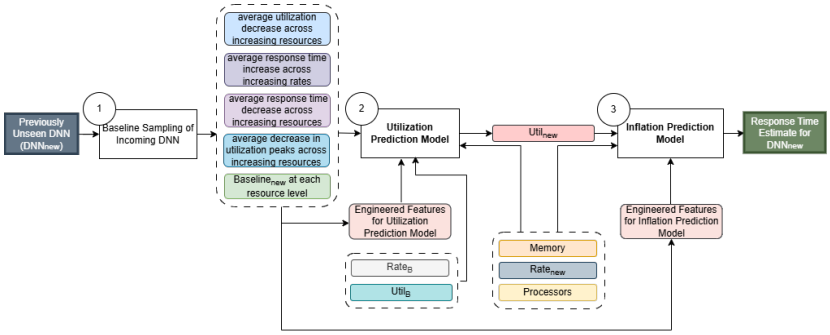

Fig. 1: Prediction pipeline for a new DNN's performance in a co-host system

co-hosted response interactions, given both DNNs' isolated profiles. Baselines collect this cursory data. We include 4 baseline features, summarizing how response time and utilization change with rate and resources:

- *Avg_Resp_Inc$_{rates}$* measures the average increase in mean response time as request rates rise. For example, from 1 to 16 requests per second across two resource settings, ResNet's average response time increases by 21.4%, while DistilBERT's increases by 52.6%, indicating higher sensitivity to rate changes. This is computed by averaging the response time increases between consecutive baseline rates across two resource settings.
- *Avg_Resp_Dec$_{res}$* records the average decrease in mean response times with an increase in resources.
- *Avg_Util_Inc$_{rates}$* records the average increase in the mean utilization across all increasing baseline rates at all baseline resource levels.
- *Avg_Peak_Dec$_{res}$* is the average decrease in the peaks of the utilization time series with increasing resources.

The baseline category of parameters also comprises a quantity referred to as *Baseline$_{new}$* for the newly arriving DNN or *Baseline$_B$* for the known DNN, which represents the isolated response time of a DNN on a possible resource setting at minimum request rate (i.e., 1 request per second).

The third set of parameters we study is *Utilization Parameters*. *Util$_{new}$* and *Util$_B$* represent the average utilization observed throughout a particular co-host deployment session for the newly arriving and previously known DNNs, respectively. They are both expressed as percentages. These parameters characterize interactions between co-hosted DNNs and are selected to support modeling and feature engineering, as detailed in Section III-B.

B. Prediction Framework

We propose a three-step prediction pipeline for response times of previously unseen co-hosted DNNs as a function of the parameters listed above. This technique is summarized in Figure 1 and comprises of three main steps:

1) Collect baseline data for the incoming *DNN$_{new}$*.
2) Predict new DNN's resource utilization using baseline features, *Rate$_{new}$*, *Rate$_B$*, allocated resources, and engineered features.
3) Predict inflation in response time due to co-hosting using the predicted utilization, baseline features, new DNN's request rate, and resource configuration.

1) Collect Baseline Data: As explained above, we use 4 baseline attributes to capture a DNN's isolated behavior across rates and resources. As identified by Shah et al. [15], baseline behavior relates to co-hosted performance; but for unseen DNNs we cannot rely on extensive baselines. To reduce cost while capturing key trends, we run a small set of baseline experiments at 5 exponentially increasing rates and two resource settings (picking one resource configuration and choosing the second by doubling the resources). This standardized setup, completed in approximately 10 minutes, captures response time and utilization trends. We also collect *Baseline$_{new}$* at this stage to later quantify the increase in a DNN's response time due to co-hosting.

2) Predict DNN$_{new}$'s Utilization: Next, we use the ML-based *Utilization Prediction Model* to predict the mean resource utilization by the newly arriving DNN in co-hosting based on allocated resources, request rates for the two DNNs, the mean utilization of the known DNN, baseline features, and engineered features capturing key interactions. These engineered features help model the highly non-linear utilization patterns that intensify under co-hosting due to resource sharing and varying request rates.

Tree-based models perform best for both prediction steps in Figure 1, as simpler models fail to capture the complex and variable nature of utilization and response time. Among them, XGBoost outperforms others in the utilization prediction step in our proposed pipeline due to its ability to capture subtle feature interactions and handle many features effectively. The model is trained on prior data from previously deployed DNNs to predict utilization for unseen ones.

3) Predict DNN$_{new}$'s Response: We input predicted *Util$_{new}$* to the *Inflation Prediction Model* by combining it with resource features. We also add two engineered features: (1) *Rate_Resp_Interact* (Eq. 1), which captures the interaction between request rate and baseline response, and (2) *Smoothened_Rate$_{new}$* (Eq. 2), which normalizes wide rate variations. These features help account for differing rate–response behaviors across DNNs and improve final predictions.

$$Rate\_Resp\_Interact = Rate_{\text{new}} \times Avg\_Resp\_Inc_{\text{rates}} \quad (1)$$

$$Smoothened\_Rate_{\text{new}} = log(Rate_{\text{new}} + 1) \quad (2)$$

These engineered features, along with predicted *Util$_{new}$*, and resource features (*Processors* and *Memory*) then work as input features that go into the *Inflation Prediction Model*. This model outputs the inflation, denoted by *Inflation$_{new}$*, in the co-hosted

DNN's response time relative to its baseline response at the same resources. *Inflation$_{new}$* is described through Eq. 3.

$$Inflation_{\text{new}} = \frac{Resp_{\text{new}}}{Baseline_{\text{new}}} \quad (3)$$

This step uses a Random Forest Regressor. Tree-based models excel in capturing complex, non-linear interactions [8], [15], with Random Forest excelling under limited features and data. Like the utilization prediction model, it is trained on historical data from previously deployed DNNs, while only brief baseline data from the new DNN (Step 1 in Figure 1) is available for prediction. This inflation in response relative to the baseline response of the new DNN (i.e., *Baseline$_{new}$*) can be used to get the predicted response time value for that DNN under the expected deployment setup using Eq. 3. Response time insights can then support informed decisions on task scheduling and ensuring SLA compliance in the system.

All response time predictions are based on average values, reflecting the training data. Although tail latencies can help estimate performance limits, they are beyond the scope of this work, as average response time sufficiently characterizes system behavior under typical workloads.

IV. Experimental Setup

A. Experiment Testbed

We conduct our experiments using a two-machine setup configured in a client-server architecture. The server is equipped with 12 Intel Xeon E5-2609 v3 processors (1.9 GHz) and 66 GB of RAM, operating on Ubuntu 22.04. Although DNNs are trained on GPUs, inference often runs on CPUs, so we use a CPU-based testbed. However, our method relies only on processors and memory, therefore it remains hardware-agnostic. Data collection is performed using TensorFlow Serving [21], a popular ML serving platform developed by Google. We use TensorFlow Serving version 2.8.2, deployed within Docker version 20.10.18. In our experiments, excluding the ones conducted in an isolated (i.e., baseline) setup, the server simultaneously hosts two DNNs, deployed in a *thin containers* setup, i.e., the two DNNs are split across two containers with a separate set of dedicated resources each. Requests are sent over Ethernet from a separate client machine. The client runs Ubuntu 22.04 on 24 Intel Xeon E5645 CPUs (2.4 GHz) with 66 GB RAM. Request rates are simulated using httperf [22] (version 0.9.1) to target TensorFlow Serving containers, with average response time as the primary metric.

B. Experiment Factors and Data Collection

To study co-hosted DNN performance, we emulate a multi-DNN deployment by varying DNNs, resources, request rates, and resulting utilizations. Our testbed runs two DNNs concurrently across multiple *Processor*, *Memory*, and *Rate* settings, collecting response times. We evaluate four diverse models: BERT [23] (language), DistilBERT [24] (language), ResNet [25] (vision), and a CNN model for classifying MNIST images [26], referred to as *MNIST-CNN*. These DNNs differ in architecture, resource needs, and input types. Although more complex Artificial Intelligence systems are becoming prevalent, the chosen DNNs for this study are still popular choices, especially in resource-constrained setups, assisting a diversity of tasks like email classification, Google's search query understanding [23], medical imaging [27], autonomous vehicles [28], digit recognition [29], and surveillance [30]. For data collection, we run 1-minute sessions on TensorFlow Serving, co-hosting two DNNs and sending requests at specified average rates following an exponential arrival distribution.

Our study uses data from three resource and request rate levels: low, medium, and high. For *Processor* values, these levels include 6, 8, and 12 processors, which correspond to half, two-thirds, and all usable processors in our system, respectively. Since the DNNs are served in thin containers, each container gets half of the total processors in that resource level (e.g., 3 dedicated processors each at the low level). *Memory* follows a similar pattern: 22 GB (low), 29 GB (medium), and 44 GB (high). *Processors* and *Memory* scale proportionally, i.e., each configuration allocates the same fraction of both resources. Workload varies via independent request rates for each DNN, creating diverse resource utilization. Generally, the utilization for the participating DNNs at a given time is kept between 15%-60% to ensure that they are operating in a stable zone.

Each experiment runs two co-located DNNs in containers on the same hardware. As described in Section III, one is designated as *DNN$_{new}$* and the other as *DNN$_B$*. We evaluate whether deployment factors (request rate, resource allocation) and *DNN$_B$*'s resource utilization can predict *DNN$_{new}$*'s utilization and response time without knowledge of either model's identity. We obtain 12 DNN pairings and 302 data records. While modest, this dataset captures a diverse range of deployment and resource configurations, which offer significant insights on co-host DNN behavior and facilitate the development of our methodology.

As described in Section III, we collect baseline features by running each DNN in isolation under varying resources and request rates. Specifically, each DNN is evaluated on 2 and 4 processors, serving 1, 2, 4, 8, and 16 requests per second (i.e., 2^0–2^4). We also record a baseline response time at low, medium, and high resource levels described in Section III-B at 1 request/sec. This is stored as $\text{Baseline}_{\text{new}}$ (and later reused as $\text{Baseline}_{\text{B}}$). It is used to compute $\text{Inflation}_{\text{new}}$, capturing the response time increase under co-hosting relative to isolation.

All non-baseline experiments were designed to produce a wide range of resource utilization values caused by each DNN to study response time inflation behavior across diverse conditions as well as to ensure our approach's predictability.

C. Metrics and Algorithms Used

We evaluate DNN performance using two metrics: *response time* and *response time inflation* (Eq. 3). While response time is used for analysis, we *predict Inflation$_{new}$* due to its empirically better predictive accuracy. The predicted inflation is combined with *Baseline$_{new}$* to estimate *Resp$_{new}$*.

Both regression models in steps 2 and 3 of Figure 1) are evaluated using *mean relative error*, which measures percentage deviation from the observed value. Both models

are fine-tuned using repeated 80–20 train/test splits with varied training sets to reduce bias. To evaluate generalization, we use four test sets where all data for a selected DNN is excluded from training, simulating a new DNN arrival and assessing cross-architectural prediction performance.

V. Results

A. Pertinent Observations and Analysis

1) Performance Similarity Based on Similar Background Utilization: Our first key finding is that a DNN_{new} exhibits similar response times at a given $Rate_{new}$, regardless of the co-hosted DNN_B, when both operate within particular respective utilization brackets. For example, DistilBERT at 5 requests per second shows consistent response times when $Util_B$ is 30–40% and $Util_{new}$ is 35–45%, with only small deviations from the *AR* in Table I. AR in this table refers to the average $Resp_{new}$ observed in this utilization bracket. This supports **RQ-1** regarding estimating response time based on resource utilization, independent of co-host DNN's (DNN_B*'s*) identity. This is a pattern observed across all tested DNNs.

However, another important behavior to note here is that these observations hold only for a fixed DNN_{new} and $Rate_{new}$: similar response times at a fixed $Rate_{new}$ and similar $Util_B$ occur *only* within the same DNN. Across *different* DNNs, response times vary significantly even under identical $Rate_{new}$, $Util_{new}$, and $Util_B$. This reflects architectural and resource differences across DNNs, showing that request rate and DNN-specific traits strongly influence performance and motivating the additional features in our unified modeling approach.

2) Impact of Baseline Features: Each DNN has distinct computational behavior, making performance prediction difficult without prior knowledge. Existing literature [15] shows that incorporating isolated baseline behavior improves co-hosted predictions. We extend this with a more efficient baseline collection method (Section III). As shown in Figure 2, DNNs exhibit markedly different utilization patterns under identical settings, implying varied co-host interactions and motivating the use of baseline parameters to better predict responses for new DNNs.

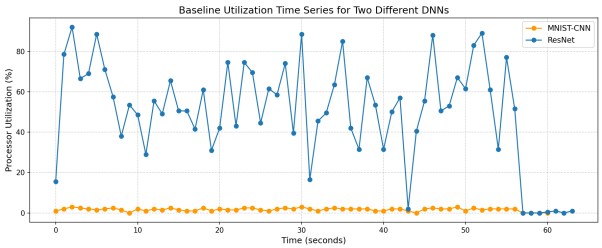

Fig. 2: Baseline utilization patterns between ResNet and MNIST-CNN running at 2 cores and 4 requests per second

B. Predicting Performance without Prior Co-hosting Data

Having established the need to incorporate utilization in **RQ-1**, **RQ-2** investigates whether performance can be predicted without prior co-hosting data. Using the baseline sampling step (Section III), we extract four features capturing isolated behavior across rates and resources, along with engineered features. We then apply a two-step pipeline: a Utilization Prediction Model to estimate $Util_{new}$, followed by an Inflation Prediction Model to predict $Inflation_{new}$ from the predicted utilization. This cascaded approach captures the relationship between resource utilization and response degradation under co-hosting. Our evaluation uses leave-one-DNN-out cross-validation, ensuring full train–test separation and simulating prediction for unseen DNNs.

The Utilization Prediction Model estimates $Util_{new}$ with a mean relative error of 23.4% across all four DNNs, limited by non-linear effects from varying architectures, resources, and request rates. This feeds into the Inflation Prediction Model, which predicts $Inflation_{new}$ with an average error of 13.6% on unseen DNNs, enabling reasonably accurate response time predictions. Despite utilization being indicative of response time, its influence decreases when combined with other features, particularly features related to rate, preventing error propagation from the Utilization Prediction Model.

Figure 3 shows the full error distribution with an average error of 8.8% for DistilBERT, 8.4% for ResNet, 15.6% for BERT, and 21.7% for MNIST-CNN. It also shows that MNIST-CNN results in the widest span of predictions, which was caused by the large variation in request rates and resource utilization values related to MNIST-CNN in our dataset.

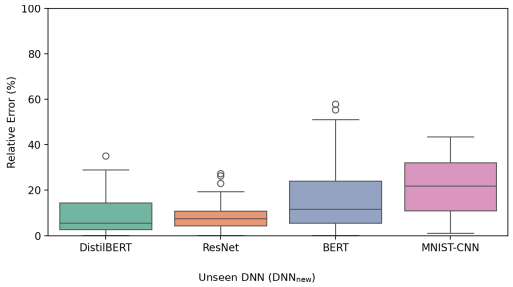

Fig. 3: Distribution of relative errors for all unseen DNNs

Figure 4 shows average relative error across resource levels per new DNN, indicating that higher resources generally lead to more unpredictable co-hosted behavior. This aligns with [15]'s observation: despite abundant resources, response time *inflation* increases. By definition (Eq. 3), *inflation* is the increase in response time under co-hosting relative to isolated performance. Thus, as isolated performance improves with more resources, co-hosted delays appear larger.

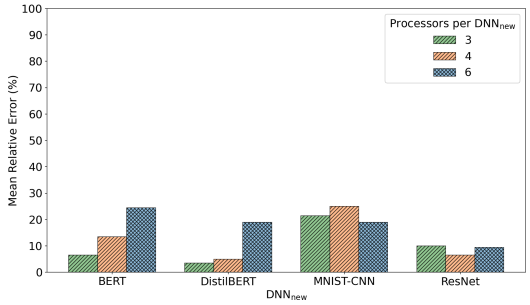

Fig. 4: Average relative error observed per resource level

Among the features, *Rate_Resp_Interact* is most influential, emphasizing the importance of request rate and baseline re-

TABLE I: DistilBERT response times with different co-host DNNs at $Util_B$ = 30-40% and $Util_{new}$ = 35-45%

DNN_{new}	$Rate_{new}$	DNN_B	$Rate_B$	$Resp_{new}$	AR	Relative Error from AR
DistilBERT	5	ResNet	4	198	201.8	2.9%
DistilBERT	5	BERT	6	206.5	201.8	3.2%
DistilBERT	5	DistilBERT	2	182.9	201.8	11.4%

sponse patterns, followed by $Util_{new}$. Resource level is least influential, though still helpful. These results suggest that in co-hosted DNN setups, administrators should prioritize regulating request rates over resources. While proper resource allocation is important for SLA compliance and efficiency, uncontrolled request rates more rapidly and significantly impact response times, increasing likelihood of SLA violations.

VI. Conclusion

This study explores predicting co-hosted DNN performance without prior data by modeling response time using minimal baseline data and expected utilization. We propose a 3-step framework and find that: (1) similar utilization alone is a weak predictor of co-hosted response, (2) small baseline experiments effectively capture computation needs, and (3) the proposed framework accurately predicts performance for unseen DNNs. These results can *support* better resource allocation, scheduling, and SLA enforcement in dynamic DNN environments by providing estimates of performance for unseen co-host DNNs. Future work will extend to diverse hardwares, more than 2 simultaneously co-hosted DNNs, and a larger diversity of DNNs, including newer architectures.

References

[1] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. Van Baalen, and T. Blankevoort, "A white paper on neural network quantization," *arXiv preprint arXiv:2106.08295*, 2021.

[2] M. Zhu and S. Gupta, "To prune, or not to prune: exploring the efficacy of pruning for model compression," *arXiv preprint arXiv:1710.01878*, 2017.

[3] H. Wu, P. Judd, X. Zhang, M. Isaev, and P. Micikevicius, "Integer quantization for deep learning inference: Principles and empirical evaluation," *arXiv preprint arXiv:2004.09602*, 2020.

[4] J. Hanhirova, T. Kämäräinen, S. Seppälä, M. Siekkinen, V. Hirvisalo, and A. Ylä-Jääski, "Latency and throughput characterization of convolutional neural networks for mobile computer vision," in *Proceedings of the 9th ACM Multimedia Systems Conference*, pp. 204–215, 2018.

[5] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, *et al.*, "Mlperf inference benchmark," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pp. 446–459, IEEE, 2020.

[6] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, and C.-J. Wu, "The vision behind mlperf: Understanding ai inference performance," *IEEE Micro*, vol. 41, no. 3, pp. 10–18, 2021.

[7] P. Ross and A. Luckow, "Edgeinsight: Characterizing and modeling the performance of machine learning inference on the edge and cloud," in *2019 IEEE International Conference on Big Data (Big Data)*, pp. 1897–1906, IEEE, 2019.

[8] S. Shah, Y. Amannejad, and D. Krishnamurthy, "Predicting the performance of dnns to support efficient resource allocation," in *2023 19th International Conference on Network and Service Management (CNSM)*, pp. 1–7, IEEE, 2023.

[9] S. Fu, S. Gupta, R. Mittal, and S. Ratnasamy, "On the use of ml for blackbox system performance prediction," in *NSDI*, 2021.

[10] B. Cai, Q. Guo, and X. Dong, "Autoinfer: A self-driving management for resource efficient, slo-aware machine learning inference in gpu clusters," *IEEE Internet of Things Journal*, 2022.

[11] Y. Li, Z. Han, Q. Zhang, Z. Li, and H. Tan, "Automating cloud deployment for deep learning inference of real-time online services," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*, pp. 1668–1677, IEEE, 2020.

[12] M. LeMay, S. Li, and T. Guo, "Perseus: Characterizing performance and cost of multi-tenant serving for cnn models," in *2020 IEEE International Conference on Cloud Engineering (IC2E)*, pp. 66–72, IEEE, 2020.

[13] J. Yang, H. Zheng, and A. Louri, "Versa-dnn: A versatile architecture enabling high-performance and energy-efficient multi-dnn acceleration," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 2, pp. 349–361, 2024.

[14] S. Ghodrati, B. H. Ahn, J. Kyung Kim, S. Kinzer, B. R. Yatham, N. Alla, H. Sharma, M. Alian, E. Ebrahimi, N. S. Kim, C. Young, and H. Esmaeilzadeh, "Planaria: Dynamic architecture fission for spatial multi-tenant acceleration of deep neural networks," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 681–697, 2020.

[15] S. Shah, Y. Amannejad, and D. Krishnamurthy, "Sla-driven performance prediction for co-hosted dnns," in *2025 IEEE/IFIP Network Operations and Management Symposium (NOMS)*, pp. 1–7, IEEE, 2025.

[16] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving dnns like clockwork: Performance predictability from the bottom up," *arXiv preprint arXiv:2006.02464*, 2020.

[17] W. Cui, M. Wei, Q. Chen, X. Tang, J. Leng, L. Li, and M. Guo, "Ebird: Elastic batch for improving responsiveness and throughput of deep learning services," in *2019 IEEE 37th International Conference on Computer Design (ICCD)*, pp. 497–505, 2019.

[18] M. Yu, Z. Jiang, H. C. Ng, W. Wang, R. Chen, and B. Li, "Gillis: Serving large neural networks in serverless functions with automatic model partitioning," in *2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS)*, pp. 138–148, 2021.

[19] C. Zhang, M. Yu, W. Wang, and F. Yan, "Enabling cost-effective, slo-aware machine learning inference serving on public cloud," *IEEE Transactions on Cloud Computing*, vol. 10, no. 3, pp. 1765–1779, 2020.

[20] M. LeMay, S. Li, and T. Guo, "Perseus: Characterizing performance and cost of multi-tenant serving for cnn models," in *2020 IEEE International Conference on Cloud Engineering (IC2E)*, pp. 66–72, 2020.

[21] "Serving Models." https://www.tensorflow.org/tfx/guide/serving/.

[22] D. Mosberger and T. Jin, "httperf—a tool for measuring web server performance," *ACM SIGMETRICS Performance Evaluation Review*, vol. 26, no. 3, pp. 31–37, 1998.

[23] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[24] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

[26] "MNIST Dataset." https://www.kaggle.com/datasets/hojjatk/mnist-dataset.

[27] S. A. Hasanah, A. A. Pravitasari, A. S. Abdullah, I. N. Yulita, and M. H. Asnawi, "A deep learning review of resnet architecture for lung disease identification in cxr image," *Applied Sciences*, 2024.

[28] Y. Li and L. Xu, "Panoptic perception for autonomous driving: A survey," *arXiv preprint arXiv:2408.15388*, 2024.

[29] S. S. Ullah, G. Li, M. Riaz, A. Ashfaq, S. Khan, and S. Khan, "Handwritten digit recognition: An ensemble-based approach for superior performance," *arXiv preprint arXiv:2503.06104*, 2025.

[30] M. Qasim and E. Verdu, "Video anomaly detection system using deep convolutional and recurrent models," *Results in Engineering*, vol. 18, p. 101026, 2023.